

Data Management

15



Whenever we purchase anything from an e-commerce website, the data list updates in their servers. This enables websites to keep track of their inventory and label items as “out of stock” when they are unavailable.



Data is used in all devices everywhere, right from smartphones to computers. Data can be in the form of text, audio, video, etc. Huge amounts of data are saved on servers every day. The data needs to be managed well to optimize the space utility, relevance and accessibility.

Data management is a process that involves collecting, authenticating, storing, protecting, and processing data. This ensures its accessibility, reliability, and security.

In this lesson, we will learn how to collect and store data so that it can be reused later.

Variables are used to store a value or a user input. For example, if a variable `x` has a value 1, then we cannot store a value 2 in the same variable. If we do, the first value will be deleted.

Consider a scenario where we need to store multiple values like birthdays of all our friends or a list of grocery items. Instead of creating multiple variables to store each value, we can use a single list in Python.

A list is an effective method to manage data.

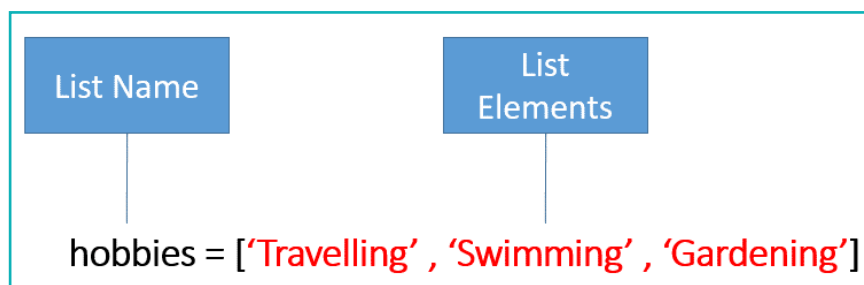


Lists in Python

- A list is a data structure in Python which can store more than 1 value.
- The items in a list are sequential. This means that every item has a specific position in the list.
- A list can contain numbers, strings or even other lists.
- A list is created by placing all items (elements) inside square brackets `[ ]` and separating them with commas.
- A list can also store duplicate values.

Let's learn more about lists by making our own.

Hobbies are an important part of our lives and it is important to have a few of them. Let's create a list to store all our hobbies there.



We can perform many operations on a list:

Print a list

We can use the `print()` statement to print a list.

```
index.py > ...
1 hobbies = ['Gardening', 'Swimming', 'Travelling']
2
3 print(hobbies)
```

```
PS D:\Python\03-CD-3-02\Programs> py index.py
['Gardening', 'Swimming', 'Travelling']
```

List index

Each element in a list has its own index number. It indicates the element's position in the list. Just like every page in a book has a page number, every element in a list has an index number.

Index numbers start from 0.

For example:

```
hobbies = ['Travelling', 'Swimming', 'Gardening']
```

Index
Number

0

1

2

We can also use negative indexing, which starts counting items from the end of the list. Negative indexing starts from -1.

```
hobbies = ['Travelling', 'Swimming', 'Gardening']
```

Index
Number

-3

-2

-1

Functions and operations on a list

1. Length of a list –Indicates the total number of items that are in a list. Use the **len()** function to determine a list's length.

Syntax: len(My_List_Name)

Let's learn about this function through an activity.

Activity 1

Determine the length of a list.

- 1 Create a list “hobbies” and store 3 items in it – ['Gardening', 'Swimming', 'Travelling'].
- 2 To determine the length of the list, use the len() function. Write **len(hobbies)**.
- 3 Store the length returned by **len(hobbies)** in another variable “**length_hobbies**”.
- 4 Finally, display the result using the **print(length_hobbies)** statement.

```
index.py > ...
1 hobbies = ['Gardening', 'Swimming', 'Travelling']
2
3 length_hobbies = len(hobbies)
4
5 print(length_hobbies)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS D:\Python\Programs> py index.py
3
```

Note:

The rest of the activities in this lesson will use the list of hobbies we've just created.

2. Add an item to a list – The `append()` function is used to add an element at the end of an existing list.

Syntax: `My_List_Name.append(Item_Name)`

Activity 2

Add a fourth hobby to the list.

- 1 To add new hobby in the list, use the `append()` function.
- 2 `hobbies.append('Reading')` adds a new hobby “Reading”, to the existing list “hobbies”.

```
index.py > ...
1 hobbies = ['Gardening', 'Swimming', 'Travelling']
2
3 hobbies.append('Reading')
4
5 print(hobbies)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS D:\Python\Programs> py index.py
3
PS D:\Python\Programs> py index.py
['Gardening', 'Swimming', 'Travelling', 'Reading']
PS D:\Python\Programs>
```

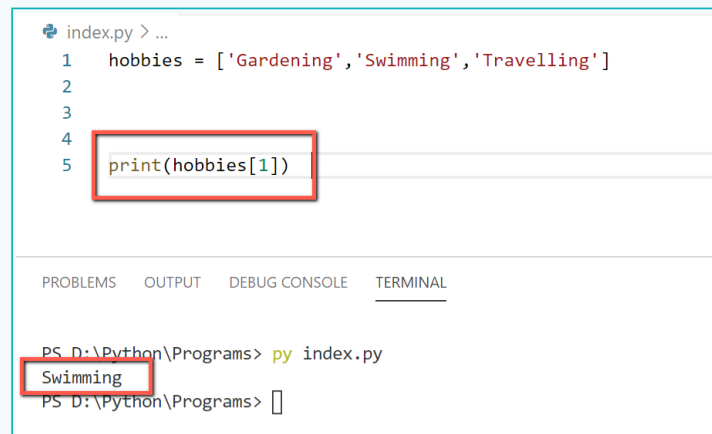
3. Access items in a list – Index number is used to access an element in a list.

Syntax: `My_List_Name[index_number]`

Activity 3

Display a specific item from a list.

- 1 To display a specific item from the existing hobbies list, use its index number.
- 2 The index number is specified within square brackets[], along with the name of the list.
- 3 hobbies[1] displays the first hobby from the list Here, hobbies is the name of the list and 1 is the index number.



```
index.py > ...
1 hobbies = ['Gardening', 'Swimming', 'Travelling']
2
3
4
5 print(hobbies[1])
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS D:\Python\Programs> py index.py
Swimming
PS D:\Python\Programs> []
```

Note:

Index numbers always start from 0. The first item in the list will have index number of 0, the second item will have index number of 1, and so on.

Activity 4

Display an item from the end of a list using negative indexing.

- 1 Use negative indexing to display items from the end of a list.
- 2 Negative indexing starts from -1, and continues with -2, -3 etc.
- 3 For example: **print(hobbies[-1])**

Activity 4

```
index.py > ...
1 hobbies = ['Gardening', 'Swimming', 'Travelling']
2
3
4 print(hobbies[-1])
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS D:\Python\Programs> py index.py
Travelling
PS D:\Python\Programs> █
```

4. Remove an item from a list – Use **del()** function to remove a single item from a list.

Syntax: **del My_List_Name[index_number]**

Activity 5

Remove an item from the hobbies list.

- 1 Delete “Travelling” from the hobbies list.
- 2 The item “Travelling” has an index number of 2. To delete this item, write **del hobbies[2]**

```
index.py > ...
1 hobbies = ['Gardening', 'Swimming', 'Travelling']
2
3 del hobbies[2]
4
5 print(hobbies)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS D:\Python\Programs> py index.py
['Gardening', 'Swimming']
```

5. Empty a list – Use `clear()` function to remove all the items from a list.

Syntax: `My_List_Name.clear()`

Activity 6

Clear the entire list of hobbies.

1 To empty the list, write `hobbies.clear()`.

```
index.py > ...
1 hobbies = ['Gardening', 'Swimming', 'Travelling']
2
3 hobbies.clear()
4
5 print(hobbies)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS D:\Python\Programs> py index.py
[]
PS D:\Python\Programs> █
```

6. Sort items in list – Use `sort()` function to sort the items in a list, either in an ascending or descending order.

Syntax: `My_List_Name.sort()`

Activity 7

Sort a list in ascending order.

1 To sort the list in ascending order, write `hobbies.sort()`.

```
1 hobbies = ['Travelling', 'Swimming', 'Gardening', 'Reading']
2
3 hobbies.sort()
4
5 print(hobbies)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS D:\Python\Programs> py index.py
['Gardening', 'Reading', 'Swimming', 'Travelling']
PS D:\Python\Programs> █
```


Activity 8

Sort a list in descending order.

- 1 To sort the list in descending order, write 'hobbies.sort()' and set the 'reverse' attribute to 'True'.

```
hobbies = ['Swimming', 'Travelling', 'Gardening', 'Reading']  
hobbies.sort(reverse = True)  
print(hobbies)  
  
['Travelling', 'Swimming', 'Reading', 'Gardening']
```

All the information in variables and lists is stored on volatile memory. This is lost when the computer shuts down or the program ends. So, reading and writing information to files in any programming language is an important feature. When data is saved to a permanent file, we can retrieve it at any other time without worrying about data loss.

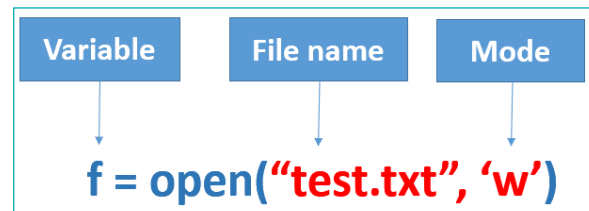
File input-output

- The named locations in a computer's memory are called files.
- When we want to read from or write to a file, we need to open it first and read the data. Then, we need to close it so that the file can be accessed by someone else.
- In Python, a file operation takes place in the following order:
 1. Open a file
 2. Read or write the file (perform operation)
 3. Close the file
- We can state the mode while opening a file. This decides whether we want to read **r**, write **w** or append **a** to the file. The default is read in text mode. In this mode, we get strings as an output after reading from the file.

File input-output

Mode	Description
r	Opens a file for reading. (default)
w	Opens a file for writing. Creates a new file if it does not exist or overwrites the file if it exists.
x	Opens a file for exclusive creation. If the file already exists, the operation fails.
a	Opens a file for appending at the end of the file. Creates a new file if it does not exist.
t	Opens a file in text mode. (default)
b	Opens a file in binary mode.
+	Opens a file for updating (reading and writing).

- **open()** command used to open a file in Python. It returns a file object as specified in the parameters. It opens the file in the background and reads or writes data in it.



For example,

```

f = open("test.txt")      # read or read in text mode
f = open("test.txt", 'w') # write in text mode
f = open("test.txt", 'a') # write in append mode
    
```

- After performing operations on a file, it needs to be closed properly.
- Closing a file frees up the resources that were tied with it. We use the `close()` command for this.

```

f = open("test.txt", 'a') # write in append mode

# perform file operations

f.close()
    
```

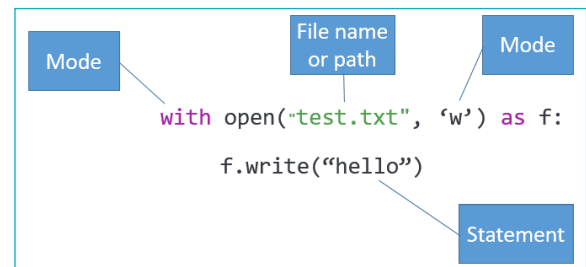
Bytes

Object : An object is a unique instance of a data structure. It consists of data members and methods.

Method : A method is an object's reusable piece of code that can be invoked/called at any point in the program.

With statement:

- “**with**” statement is a built-in read or write statement for a file in Python.
- “**with**” statement ensures that the file closes when the block inside the with statement is exited.
- It does not need to call the **close()** function.



Exercise

Tick mark ☒ the correct answer

1 Which of the following lists is correct?

- a. hobbies = 'Travelling', 'Swimming', 'Gardening' ☐
- b. hobbies = ('Travelling', 'Swimming', 'Gardening') ☐
- c. hobbies = ['Travelling', '2', 'Gardening'] ☐
- d. None of the above ☐

2 Which of the following will be used to add an element in an existing list?

- a. len() ☐
- b. write() ☐
- c. del ☐
- d. append() ☐

Exercise

 **State whether True or False**

3 A list can only contain numbers.

 _____

4 Every element in a list has an index number associated with it, which indicates the position of the element in the list.

 _____

5 In a 'with' statement, it is necessary to call the close() function.

 _____

Activity 9

 **Create a program that will add two numbers and display the result.**

1 Open VS Code application and create a new file through **File menu >> New File**.

2 Save the file through **File >> Save**.

3 Name the file **Hobbies** and set the extension as **.py**, which indicates that it's a Python program. This creates a file "**Hobbies.py**", which is visible in the VS Code Sidebar.

Activity 9

- 4 Create a list “hobbies” and add elements as shown:

```
Hobbies.py > ...  
1 hobbies = ['Travelling', 'Swimming', 'Gardening']
```

- 5 Open a text file “Hobbies.txt” in write mode as a file object “f” using “with” statement. If no file exists with the given file name, then a new file with that name will be created.

```
1 hobbies = ['Travelling', 'Swimming', 'Gardening']  
2  
3 with open("Hobbies.txt", 'w') as f:  
4 |
```

Write() method can only accept string data type as an input. We cannot input a list directly to the write() method of file object “f”. For each list element to be an input for write() method, we can use ‘for’ loop.

- 6 In each iteration of the for loop, extract one element from the list and store it in a text file using write() method as shown:

```
Hobbies.py > ...  
1 hobbies = ['Travelling', 'Swimming', 'Gardening']  
2  
3 with open("Hobbies.txt", 'w') as f:  
4 |  
5 | for item in hobbies:  
6 | | f.write('%s\n' % item)  
7 |
```

Where,

- **%s** signifies that we want to add a string value into another string.
- **% item** is where we specify which values should be added to the string.

Activity 9

- 7 To execute the program, use the inbuilt terminal in VS Code. This can be accessed by clicking on **Terminal >> New Terminal**.
- 8 To execute the program type **py Hobbies.py** in the terminal and press **Enter**.

A new file named “**Hobbies.txt**” is generated as an output. It will contain list elements as shown:

```
Hobbies.txt
1 Travelling
2 Swimming
3 Gardening
```

In this lesson, we learnt about data management and how it is important. To manage data, we have to save it and keep it updated. To store data, we used the List function in Python. We used activities to understand operations for lists in Python. To permanently store data in a computer, we use a file system. With “r”, “w” and “a”, data can be read from, written to, and appended to the file.

Exercise

 **Answer in one line**

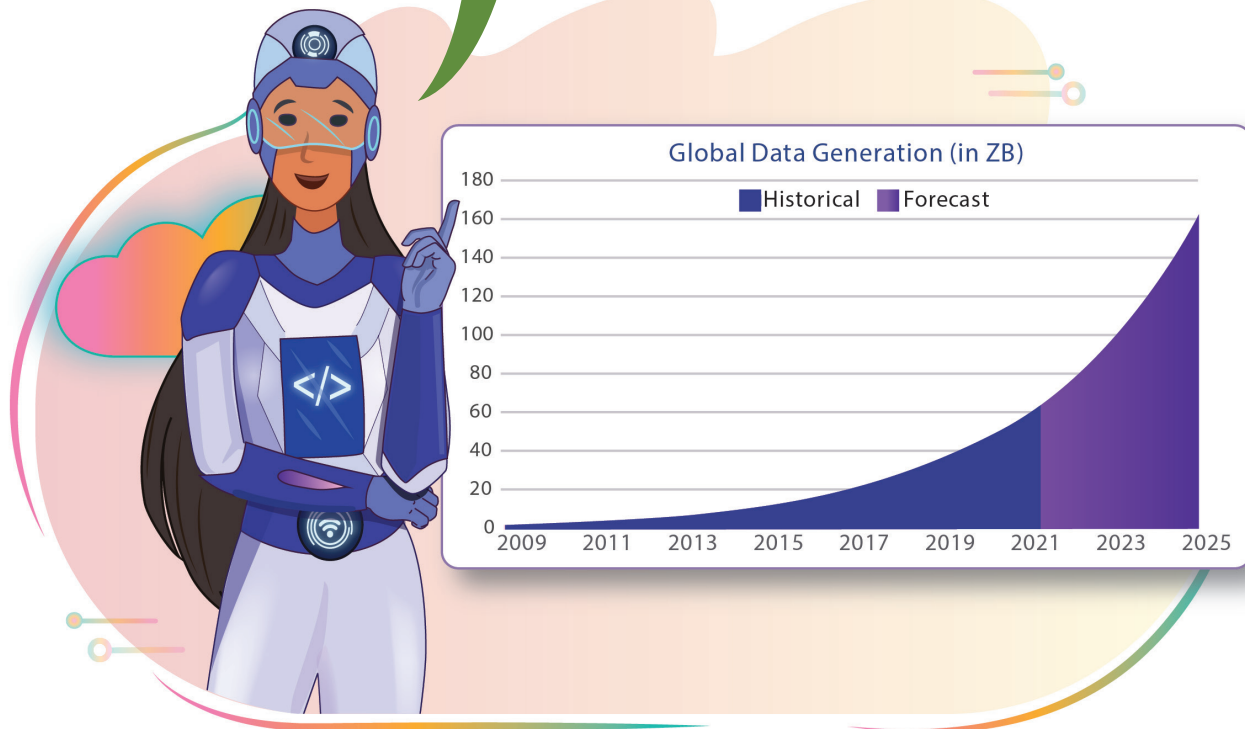
- 6 What is the syntax to access list items in ascending order?



- 7 What is the function of append mode in file?



By 2025, there will be 175 zettabytes of data in the global datasphere. Hence, data management is gaining more and more importance.



Tech Flash

- **Data management** : A process that includes collecting, authenticating, storing, protecting, and processing data to ensure its accessibility, reliability, and security.
- **List** : A data structure in Python which stores more than one value.
- **Object** : A unique instance of a data structure. An object consists of data members and methods.
- **Method** : A reusable piece of object code that can be invoked or called at any point in the program.